

BINARY SEARCH ON ANSWER

```
int lo=minPossible, hi=maxPossible, ans=-1;
while (lo <= hi) {
    int mid = lo + (hi-lo)/2;
    if (isValid(mid)) { ans=mid; lo=mid+1; } // mid works, try bigger
    else hi = mid - 1; // mid fails, try smaller
}
```

Demo: minimum max value when splitting array into k parts

```
bool canSplit(vector<int>& a, int maxSum, int k) {
    int parts=1; long long cur=0;
    for (int x : a) { if (cur+x > maxSum) { parts++; cur=x; } else cur+=x; }
    return parts <= k;
}

int lo=*max_element(a.begin(),a.end()), hi=accumulate(a.begin(),a.end(),0), ans=hi;
while (lo <= hi) {
    int mid = lo + (hi-lo)/2;
    if (canSplit(a, mid, k)) { ans=mid; hi=mid-1; } else lo=mid+1;
}
```

Demo: smallest x such that $x*x \geq n$

```
long long lo=0, hi=n, ans=n;
while (lo <= hi) {
    long long mid = lo + (hi-lo)/2;
    if (mid*mid >= n) { ans=mid; hi=mid-1; } else lo=mid+1;
}
```

GCD / LCM

```
__gcd(a, b); // built-in GCD (also gcd(a,b) in C++17+)
long long lcm(long long a, long long b) { return a / __gcd(a,b) * b; } // divide
first, avoid overflow
```

How GCD actually works – iterative Euclidean Algorithm

```
long long gcdManual(long long a, long long b) {
    while (b != 0) { long long temp=b; b=a%b; a=temp; }
    return a;
}

long long g=a[0]; for(int i=1;i<n;i++) g=__gcd(g,a[i]); // GCD of array
long long l=a[0]; for(int i=1;i<n;i++) l=l/__gcd(l,a[i])*a[i]; // LCM of array
```

Demo: simplify fraction a/b

```
int g = __gcd(a, b); a /= g; b /= g;
```

===== GCD/LCM USEFUL FACTS =====

```
gcd(a, b) * lcm(a, b) == a * b; // always true – useful identity
gcd(a, 0) == a; // base case fact
gcd(a, b) == gcd(b, a % b); // Euclidean recurrence itself
// a and b are coprime (share no common factor) if gcd(a,b) == 1
bool coprime = (__gcd(a, b) == 1);
// n numbers are pairwise "reducible" to gcd=1 doesn't mean pairwise coprime
// (e.g. gcd(6,10,15)=1 but no two of them are individually coprime)
// divisibility check using gcd
bool aDividesB = (b % a == 0); // simple check, no gcd needed
// sum of two fractions a1/b1 + a2/b2, simplified
// numerator = a1*b2 + a2*b1, denominator = b1*b2, then divide both by gcd
long long num = a1*b2 + a2*b1, den = b1*b2;
long long g = __gcd(num, den); num/=g; den/=g;
```